



Aplicaciones e Industria de IoT

Semana 7 — Salud Conectada, Seguridad y Ética en IoT

KOICA

IGU HANDONG GLOBAL
UNIVERSITY



UNA

ÍNDICE

- 1. Salud Conectada**
- 2. Amenazas y Vulnerabilidades en IoT**
- 3. Medidas de Protección y Buenas Prácticas**
- 4. Privacidad, Ética y Marco Regulatorio**
- 5. Casos de Éxito y Lecciones Aprendidas**
- 6. Laboratorio PBL: Plataforma IoT Segura para Monitoreo de Pacientes**
- 7. Resumen, Bibliografía y Próximos Pasos**



Repaso de la Semana 6 y Objetivos de la Semana 7



Lo que aprendimos en la Semana 6

- Agricultura de precisión: GPS, sensores, drones y satélites.
- Conectividad rural: LoRaWAN vs NB-IoT en grandes extensiones.
- Logística: visibilidad E2E, geofencing, telemática.
- Cadena de frío y trazabilidad con QR, RFID y blockchain.
- Casos: John Deere, DHL, Walmart, Maersk.



Objetivos de la Semana 7

- Conocer aplicaciones de IoT en salud (eHealth, wearables, RPM).
- Identificar amenazas y vulnerabilidades en dispositivos IoT.
- Aplicar medidas de protección: cifrado, autenticación, segmentación.
- Analizar implicancias éticas: privacidad, vigilancia, sesgos.
- Implementar un pipeline IoT completo con HiveMQ y base de datos.
- Construir matriz de riesgos para el proyecto integrador.



Conexión: Después de explorar 6 verticales, hoy nos enfocamos en lo crítico: salud, seguridad y ética

4

Aplicación
HOY: Salud Conectada

3

Plataforma
HOY: BD + Analítica

2

Red
HOY: Cifrado y TLS

1

Dispositivo
HOY: Wearables seguros

1. SALUD CONECTADA

¿Qué es la Salud Conectada?

La salud conectada (eHealth, mHealth, telemedicina) utiliza IoT, wearables y plataformas digitales para monitorear pacientes de manera remota y continua. Permite detectar problemas tempranos, gestionar enfermedades crónicas y reducir consultas innecesarias. El mercado global se estima en USD 660.000 millones para 2030 (Grand View Research).

♥ Los 4 Pilares de la Salud Conectada



Wearables — Los Sensores que Llevamos Encima

Tipos de Wearables y Variables que Monitorean

1 SMARTWATCH



Monitorea ritmo cardíaco, ECG, SpO2, pasos, sueño, caídas y GPS.



2 PULSERA DE ACTIVIDAD



Mide pasos, calorías, ritmo cardíaco y calidad del sueño.



3 PARCHES BIOMÉTRICOS

ECG continuo, frecuencia respiratoria y temperatura corporal.



4 MONITOR CONTINUO DE GLUCOSA (CGM)



Mide glucosa intersticial cada pocos minutos.



5 ROPA INTELIGENTE



Monitorea frecuencia respiratoria, ECG, postura y movimiento.



6 ANILLOS INTELIGENTES

Miden temperatura corporal, HRV, sueño y estrés.



Los wearables de salud complementan el cuidado médico, pero no sustituyen un diagnóstico profesional.



Monitoreo Remoto de Pacientes (RPM) – Hospital en Casa

El RPM (Remote Patient Monitoring) consiste en dispositivos médicos conectados que el paciente usa en su hogar, transmitiendo signos vitales al equipo de salud. Reduce hospitalizaciones, mejora adherencia al tratamiento y permite intervención temprana. La pandemia de COVID-19 aceleró su adopción mundial 10-15 años.



Aplicaciones Clínicas del RPM

Enfermedades cardíacas

Tensiómetro, ECG, oxímetro

Tensiómetros conectados, ECG portátiles, marcapasos con telemetría. Reducción de readmisiones hospitalarias del 30-50%.

Diabetes

CGM, glucómetro, bomba insulina

Glucómetros y bombas de insulina conectados. Ajuste automático de dosis. Mejor control glicémico, menos complicaciones.

Enfermedad respiratoria (EPOC, asma)

Oxímetro, espirómetro

Espirómetros conectados, oxímetros, inhaladores con sensor. Detección temprana de exacerbaciones.

Adultos mayores

Acelerómetro, GPS, alerta

Sensores de caída, monitoreo de actividad, alertas SOS. Permite envejecer en casa con seguridad.

Dispositivos Médicos Críticos Conectados

Equipos Médicos con Conectividad IoT

1 Marcapasos y desfibriladores



Telemetría inalámbrica.
Reportan eventos
arritmicos.
Vida útil: 5-15 años.



2 Bombas de infusión



Dosificación
programada.
Supervisión y
ajuste remoto.



3 Ventiladores mecánicos



Monitoreo remoto
de parámetros
respiratorios.
Integración con HIS.



Datos en
tiempo real



4 Equipos de imagen



Mantenimiento
predictivo.
Telediagnóstico.
Transmisión de
imágenes.



2. AMENAZAS Y VULNERABILIDADES

¿Por Qué los Dispositivos IoT son Vulnerables?

Los dispositivos IoT enfrentan desafíos de seguridad únicos: recursos limitados, ciclos de vida largos, despliegue masivo y acceso físico relativamente sencillo. Según Sicari et al. (2015), la seguridad en IoT es fundamentalmente diferente a la de redes corporativas tradicionales.

Las 6 Vulnerabilidades Estructurales del IoT

Recursos limitados

Microcontroladores con poca memoria y CPU. Difícil implementar cifrado fuerte (AES-256, TLS 1.3).

Firmware desactualizado

Muchos fabricantes no actualizan. Vulnerabilidades persisten por años. Sin gestión OTA segura.

Credenciales por defecto

Contraseñas 'admin/admin' o 'root/12345'. Mirai botnet explotó esto para tumbar Dyn en 2016.

Comunicación sin cifrar

Muchos dispositivos usan MQTT/HTTP plano. Sniffing trivial con Wireshark en la misma red.

Acceso físico

Sensores en campo, en lugares públicos. Posible extracción de firmware o claves del chip.

Cadena de suministro

Componentes de múltiples proveedores. Una sola librería vulnerable compromete todo el dispositivo.

Tipos de Ataques a Dispositivos y Redes IoT

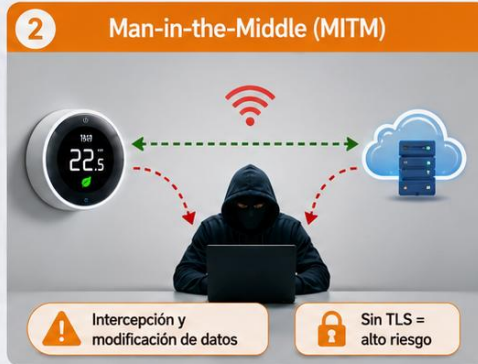
Los 6 Vectores de Ataque más Frecuentes

1 Botnet (Mirai)



Millones de dispositivos secuestrados
Ataques DDoS masivos

2 Man-in-the-Middle (MITM)



Intercepción y modificación de datos

Sin TLS = alto riesgo

3 Replay attack



Mensaje legítimo
10101010

Mensaje reenviado
10101010

Reenvío de mensajes válidos
Ataque por repetición

4 Firmware reverso



Análisis del firmware
Extracción de credenciales y claves

5 Side-channel attack



Inferencia a partir de consumo de energía, tiempo o emisiones electromagnéticas

6 Ransomware en IoT



Dispositivos bloqueados
Exigen rescate para restablecer el servicio

Modelo STRIDE – Cómo Analizar Amenazas de Seguridad

STRIDE es un modelo de amenazas creado por Microsoft en 1999, hoy estándar en ciberseguridad. Cada letra representa una categoría de amenaza. Permite analizar sistemáticamente los riesgos de un sistema IoT y definir contramedidas.

Los 6 Elementos del Modelo STRIDE

S

Spoofing

Suplantación de identidad. Un atacante se hace pasar por otro dispositivo o usuario legítimo.

✓ Autenticación fuerte, certificados X.509

T

Tampering

Manipulación de datos. Alteración no autorizada de información en tránsito o almacenada.

✓ Hash, firmas digitales, TLS

R

Repudiation

Negación de acciones. Un usuario niega haber hecho algo que sí hizo.

✓ Logs firmados, auditoría, timestamps

I

Information Disclosure

Divulgación de información. Exposición no autorizada de datos sensibles.

✓ Cifrado en tránsito y en reposo

D

Denial of Service

Denegación de servicio. Hacer que un sistema deje de funcionar para usuarios legítimos.

✓ Rate limiting, redundancia, firewall

E

Elevation of Privilege

Escalada de privilegios. Un usuario obtiene permisos mayores a los autorizados.

✓ Mínimo privilegio, separación de roles

Ejercicio E2E — Sensor → MQTT → ThingsBoard Cloud → CSV

🔥 EJERCICIO EN CLASE — 30 MIN

🎯 **Objetivo:** Probar el flujo IoT completo, almacenar en ThingsBoard y exportar los datos a CSV

📋 Instrucciones Paso a Paso

1. Crear cuenta gratuita en [thingsboard.cloud](https://thingsboard.io) (o usar demo.thingsboard.io). Plan free disponible.
2. Crear un Device → copiar su Access Token.
3. En Wokwi: ESP32 + DHT22 (GPIO4) + LED (GPIO25).
4. Copiar el código de la siguiente diapositiva.
5. Reemplazar TOKEN con el de tu dispositivo.
6. Ejecutar y verificar en "Latest Telemetry" del device.
7. Crear un Dashboard con widget "Timeseries Line Chart".
8. Agregar widget "Timeseries Table" → botón Export to CSV.

💡 ¿Qué Van a Aprender?

- Pipeline IoT completo de extremo a extremo.
- MQTT con TLS (puerto 8883) y autenticación por token.
- Almacenamiento y dashboards nativos en ThingsBoard.
- Exportación de datos históricos a CSV para análisis.

🔑 Conexión ThingsBoard Cloud

HOST: `thingsboard.cloud`
PORT: `8883 (TLS)` · USER: `Access Token`
PASS: `vacío` · TOPIC: `v1/devices/me/telemetry`

Ejercicio • Código Arduino IDE – Cliente MQTT a ThingsBoard



Código Completo – Reemplazar el Access Token con el tuyo

```
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <PubSubClient.h>
#include <DHT.h>
#define DHTPIN 4
#define DHTTYPE DHT22
#define LED 25

// == CREDENCIALES – REEMPLAZAR ==
const char* ssid = "Wokwi-GUEST";
const char* pass = "";
const char* tb_host = "thingsboard.cloud";
const int tb_port = 8883; // TLS
const char* tb_token = "TU_ACCESS_TOKEN";
// del Device
// En ThingsBoard: token = usuario MQTT,
password vacío
const char* topic =
"v1/devices/me/telemetry";

WiFiClientSecure espClient;
PubSubClient mqtt(espClient);
DHT dht(DHTPIN, DHTTYPE);
int seq = 0;

void setup() {
  Serial.begin(115200); dht.begin();
  pinMode(LED, OUTPUT);
  WiFi.begin(ssid, pass);
  while(WiFi.status() != WL_CONNECTED)
    delay(500);
  Serial.println("WiFi OK");
  espClient.setInsecure(); // En Wokwi
  (sin CA propia)
  mqtt.setServer(tb_host, tb_port);
}

void reconectar() {
  while(!mqtt.connected()) {
    Serial.print("Conectando a
ThingsBoard...");
    // Usuario = token, contraseña vacía
    if(mqtt.connect("wearable-01",
tb_token, "")) {
      Serial.println(" OK!");
    } else {
      Serial.print(" Error: ");
      Serial.println(mqtt.state());
      delay(3000);
    }
  }
}

void loop() {
  if(!mqtt.connected()) reconectar();
  mqtt.loop();
  float t = dht.readTemperature();
  float h = dht.readHumidity();
  if(isnan(t)) { delay(2000); return; }

  // Simular ritmo cardiaco (60-100 bpm
normal)
  int bpm = 60 + random(0, 40);
  // Simular SpO2 (95-100% normal)
  int spo2 = 95 + random(0, 5);
  seq++;

  // Encender LED si valores fuera de rango
  if(bpm < 60 || bpm > 100 || spo2 < 95) {
    digitalWrite(LED, HIGH);
  } else {
    digitalWrite(LED, LOW);
  }

  // JSON: cada clave se vuelve una
variable de
// telemetría en ThingsBoard
automáticamente
  char json[200];
  snprintf(json, 200,
    "{\"temp\":%.1f,\"hum\":%.1f,\"
    \"bpm\":%d,\"spo2\":%d,\"seq\":%d}\",
    t, h, bpm, spo2, seq);
  mqtt.publish(topic, json);
  Serial.println(json);
  delay(5000);
}
```

Notas Clave

Autenticación por token

`mqtt.connect(id, token, "")`

En ThingsBoard el Access Token va como usuario MQTT y la contraseña queda vacía.

Topic fijo

`v1/devices/me/telemetry`

No se cambia. Cada clave del JSON se convierte en una variable de telemetría.

Puerto 8883 (TLS)

Cifrado obligatorio para datos médicos (HIPAA, LGPD). `setInsecure()` solo para Wokwi; en producción usar `setCACert()`.

Exportar a CSV

En el Dashboard, el widget "Timeseries Table" tiene un botón para descargar los datos históricos en formato CSV.

3. MEDIDAS DE PROTECCIÓN

Las 7 Medidas Mínimas de Protección IoT



Checklist Esencial de Seguridad para Cualquier Despliegue IoT

1 Cambiar credenciales por defecto



Contraseñas únicas y robustas.

2 Cifrar comunicaciones (TLS)



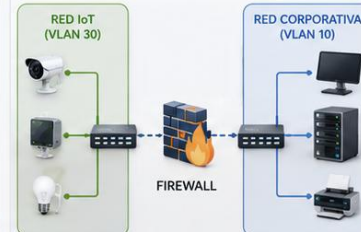
Datos protegidos en tránsito.

3 Autenticar cada dispositivo



Identidad única por dispositivo.

4 Segmentar la red IoT



Aislamiento por segmentos.

5 Actualizaciones OTA seguras



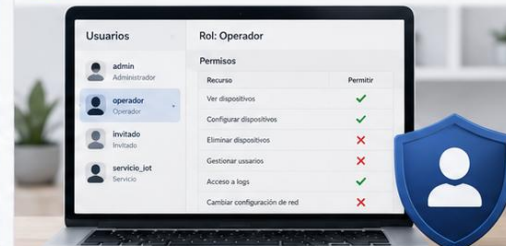
Firmware firmado y rollback.

6 Registrar y auditar (logs)



Trazabilidad y detección de anomalías.

7 Principio de mínimo privilegio



Solo acceso necesario.

Cifrado y Autenticación – La Primera Línea de Defensa

Cifrado en IoT

TLS / SSL

Transport Layer Security. Cifra la comunicación entre dispositivo y servidor. Estándar moderno: TLS 1.3.

AES (Advanced Encryption Standard)

Cifrado simétrico para datos almacenados. AES-256 es el estándar industrial.

DTLS (Datagram TLS)

Variante de TLS para protocolos UDP. Usado con CoAP en dispositivos con poca energía.

Hash funciones

SHA-256, SHA-3 para integridad de mensajes y firmas digitales. Garantizan que los datos no fueron alterados.

Autenticación

Username + Password

Lo más básico. Funciona si las contraseñas son fuertes y únicas por dispositivo. Usado en MQTT cloud.

API Tokens / JWT

Tokens firmados criptográficamente con tiempo de expiración. Revocables sin cambiar contraseñas.

Certificados X.509

Cada dispositivo tiene su propio certificado único. Estándar para entornos empresariales (AWS IoT, Azure IoT).

OAuth 2.0 / OpenID Connect

Para integración con apps de usuarios. Permite delegar autorización sin compartir credenciales.

 **Regla de oro: nunca hardcodear credenciales en el código fuente. Usar variables de entorno o secretos.**

Segmentación de Red y Gestión de Identidades IoT

La arquitectura Zero Trust aplicada a IoT: ningún dispositivo confiable por defecto. Segmentación de red, gestión de identidades únicas y monitoreo continuo. Asume que cualquier dispositivo puede ser comprometido.



Arquitectura de Red Segura para IoT

DMZ IoT

Zona desmilitarizada donde viven los gateways IoT. Aislada de la red corporativa interna.

VLAN dedicada

VLAN separada por tipo de dispositivo: cámaras en una, sensores en otra, médicos en otra.

Firewall por capa

Reglas estrictas: solo tráfico necesario (puerto 8883 al broker MQTT, por ejemplo). Bloqueo del resto.

PAM y CMDB

Privileged Access Management + Configuration Management DB. Inventario completo de cada dispositivo y sus accesos.

4. PRIVACIDAD, ÉTICA Y MARCO REGULATORIO

Privacidad de Datos — Información Sensible en IoT

Los dispositivos IoT generan datos muy sensibles: ubicación de personas, hábitos de consumo, signos vitales, conversaciones en el hogar. Weber (2010) advertía: el desafío no es solo proteger los datos, sino decidir qué se debe recolectar y por cuánto tiempo guardarlos.



Los 4 Principios de Privacidad en IoT

Minimización

Recolectar solo los datos estrictamente necesarios para la función del dispositivo. Si no se usa, no se guarda.

Transparencia

El usuario debe saber qué datos se recolectan, cómo se usan y con quién se comparten. Política clara y comprensible.

Consentimiento

Activo, informado y reversible. El usuario puede retirar el consentimiento en cualquier momento.

Anonimización

Disociar datos personales del identificador. Análisis agregados sin posibilidad de re-identificación.

Marco Regulatorio – GDPR, LGPD y Normativa Local



Las Tres Regulaciones Más Relevantes para IoT

GDPR (Europa)

Mayo 2018

General Data Protection Regulation. La regulación más estricta del mundo. Multas hasta 4% de facturación global. Derecho al olvido, portabilidad, consentimiento explícito. Aplica a cualquier empresa que procese datos de europeos.

LGPD (Brasil)

Agosto 2020

Lei Geral de Proteção de Dados. Inspirada en GDPR. Adoptada en Brasil con multas hasta 2% de facturación. Estableció ANPD (Autoridade Nacional). Referente para Latinoamérica.

Ley 6534/20 (Paraguay)

Noviembre 2020

Ley de Protección de Datos Personales Crediticios. Paraguay aún no tiene ley general equivalente a GDPR pero hay anteproyecto. Habeas data en el art. 135 de la Constitución protege a las personas.

Vigilancia, Sesgos Algorítmicos e Impacto Social

Vigilancia y Uso Secundario

El riesgo de la vigilancia masiva:

Cámaras urbanas + reconocimiento facial + tracking de smartphones permiten construir perfiles de cada ciudadano. China lo aplica con su Social Credit System.

Uso secundario de datos:

Datos recolectados con un propósito se usan para otro. Ejemplo: Strava reveló bases militares secretas al publicar mapas de calor de actividad deportiva (2018).

Sesgos Algorítmicos

¿Qué es?

Discriminación causada por modelos de IA entrenados con datos sesgados. Sistemas de IoT con IA pueden perpetuar o amplificar prejuicios.

Ejemplos:

Oxímetros menos precisos en pieles oscuras (caso documentado en COVID-19). Algoritmos de contratación que discriminan a mujeres. Reconocimiento facial con mayor tasa de error en personas no blancas.

Mitigación: diversidad en datos de entrenamiento, auditoría continua, equipos diversos.



Impacto ambiental: cada dispositivo IoT tiene huella de carbono. Repensar ciclo de vida y residuos electrónicos.

5. CASOS DE ÉXITO Y LECCIONES APRENDIDAS

Caso 1: Apple Watch – Detección Temprana de Salud

 Apple Watch se ha convertido en el wearable de salud más vendido del mundo, con más de 100 millones de unidades. Incorpora sensores médicos avanzados (ECG, SpO2, ritmo cardíaco, temperatura, detección de caídas). Su impacto va más allá de fitness: ha salvado miles de vidas detectando arritmias y caídas en adultos mayores. Apple Health permite compartir datos con el médico de cabecera.

Tecnologías Aplicadas

- ECG de derivada única, validado por FDA en 2018.
- Detección de fibrilación auricular con sensibilidad 98%.
- Sensor SpO2 desde Series 6 para oxigenación sanguínea.
- Detección de caídas con acelerómetro y giroscopio.
- Integración con Apple Health Records (HL7 FHIR).

Impacto en Salud Pública

- Apple Heart Study (Stanford 2019): 419.000 participantes.
- Múltiples casos documentados de detección temprana de fibrilación auricular y arritmias.
- Detección de caídas con llamada de emergencia automática.
- Datos anónimos contribuyen a estudios epidemiológicos.
- Privacidad: procesamiento on-device, cifrado end-to-end.

Caso 2: Philips HealthSuite – Plataforma Hospitalaria



Philips HealthSuite es una plataforma cloud que conecta dispositivos médicos, sistemas de información hospitalaria (HIS) y aplicaciones de salud personal. Implementada en más de 800 hospitales globales, gestiona datos de millones de pacientes. Permite vista unificada del paciente, telemedicina y analítica predictiva. Compatible con HL7 FHIR y DICOM (imagen médica).

Componentes Clave

- IntelliVue: monitores de paciente con conectividad inalámbrica.
- eICU Program: telemedicina para unidades de cuidados intensivos.
- VitalMinds: análisis con IA para predicción de deterioro clínico.
- Integración con HIS, PACS, EMR existentes.
- Cumple HIPAA (USA), GDPR (Europa) y normativas locales.

Resultados Clínicos

- Reducción de mortalidad en UTI: hasta 30% con eICU.
- Reducción de tiempo de internación: 15-25%.
- Detección temprana de sepsis: 4-6 horas antes que método tradicional.
- Especialistas remotos atienden múltiples hospitales rurales.
- Ahorro de costos operativos: 10-20% anual.

Caso 3: Mirai Botnet – Lección sobre Inseguridad Masiva

⚠ En octubre de 2016, el botnet Mirai infectó más de 600.000 dispositivos IoT (cámaras IP, routers, DVRs) explotando credenciales por defecto. El 21 de octubre, dirigió un ataque DDoS masivo contra el proveedor de DNS Dyn, tumbando Twitter, Netflix, Spotify, Reddit, GitHub y CNN durante varias horas en EE.UU. y Europa. Fue el ataque DDoS más grande hasta ese momento (1.2 Tbps).

Cómo Funcionó

- Escaneó internet buscando dispositivos con puertos Telnet/SSH abiertos.
- Probó lista de 60 credenciales comunes (admin/admin, root/123).
- Una vez dentro, descargaba el malware y se reproducía.
- Cada dispositivo infectado generaba tráfico HTTP/DNS contra el objetivo.
- Código abierto: hoy hay más de 60 variantes de Mirai en circulación.

Lecciones Aprendidas

- Credenciales por defecto = vulnerabilidad masiva.
- Dispositivos olvidados son la mayor superficie de ataque.
- Regulación: la UE publicó la Ley de Ciberresiliencia (2024).
- Fabricantes ahora exigen cambio de contraseña en primer arranque.
- Segmentación de red IoT pasó a ser estándar empresarial.

Caso 4: St. Jude Medical – Cuando un Hack Puede Matar

 En 2017, la FDA (administración de alimentos y medicamentos de EE.UU.) emitió la primera alerta de retiro por ciberseguridad: 465.000 marcapasos de St. Jude Medical (hoy Abbott) tenían vulnerabilidades que permitían a un atacante remoto agotar la batería del implante o modificar el ritmo cardíaco. Los pacientes debían acudir al hospital para actualización de firmware. Fue un punto de inflexión para la regulación de dispositivos médicos.

Vulnerabilidades Encontradas

- Comunicación RF sin cifrar entre marcapasos y monitor doméstico.
- Autenticación débil: clave compartida y reutilizada.
- Comandos no autenticados podían reprogramar el dispositivo.
- Sin mecanismo de actualización segura del firmware.
- Posible ataque desde 7-15 metros con equipo SDR (USD 100).

Cambios Regulatorios

- FDA emitió guías obligatorias de ciberseguridad médica.
- SBOM (Software Bill of Materials) obligatorio para dispositivos médicos.
- Pruebas de penetración obligatorias antes de aprobación.
- Política de divulgación responsable de vulnerabilidades.
- Hoy todos los implantes médicos modernos usan TLS y firma de firmware.

6. LABORATORIO PBL

LAB PBL – Plataforma IoT Segura para Monitoreo de Pacientes

EN EQUIPOS

ESCENARIO: Un hospital necesita monitorear pacientes ambulatorios con cardiopatías

Un hospital de Asunción quiere implementar un sistema de monitoreo remoto para 50 pacientes con cardiopatías crónicas. Cada paciente lleva un wearable que mide ritmo cardíaco, oxigenación y temperatura. Los datos deben llegar al hospital con cifrado TLS, almacenarse en una base de datos cloud, y disparar alertas automáticas al equipo médico si hay valores fuera de rango. Su equipo debe diseñar e implementar el pipeline completo.

Entregables del LAB

1. Wokwi: ESP32 + DHT22 simulando wearable, publicando con MQTT+TLS.
2. HiveMQ Cloud: cluster con autenticación y topic dedicado.
3. Datacake o InfluxDB: persistencia + dashboard de signos vitales.
4. Lógica de alerta: bpm/spo2 fuera de rango → notificación.
5. Matriz de riesgos STRIDE de la solución.

Criterios de Evaluación

- Pipeline E2E funcionando
- Uso correcto de TLS y autenticación
- Dashboard con datos en vivo
- Matriz de riesgos STRIDE
- Análisis ético del sistema

LAB PBL – Guía de Desarrollo Paso a Paso

Diseñar arquitectura

5 min

Dibujar diagrama E2E: wearable ESP32 → MQTT TLS → HiveMQ Cloud → Datacake → médico. Justificar cada elección.

Configurar HiveMQ Cloud

10 min

Crear cuenta gratuita. Crear cluster (free hasta 100 conexiones). Crear usuario MQTT con credenciales fuertes. Anotar HOST.

Implementar en Wokwi

15 min

ESP32 + DHT22 + 2 LEDs (verde OK, rojo alerta). Cliente MQTT TLS conectado a HiveMQ. Telemetría cada 5s.

Configurar Datacake

10 min

Crear cuenta free. Crear device tipo MQTT. Configurar suscripción al topic. Crear dashboard con gráficos de bpm/spo2/temp.

Análisis Python

10 min

Notebook en Colab: cargar histórico, detectar anomalías (bpm > 100 ó spo2 < 95), generar reporte con tabla de eventos.

Matriz STRIDE

5 min

Identificar 1 amenaza por cada categoría STRIDE en la solución. Proponer mitigación. Documentar en una tabla.

Análisis ético

3 min

Reflexión: ¿qué datos NO se deberían recolectar? ¿cómo se anonimizan? ¿quién accede? ¿cuándo se borran?

LAB PBL • Código Arduino IDE – Wearable Médico Seguro



Código Arduino – MQTT con TLS y lógica de alerta

```
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <PubSubClient.h>
#include <DHT.h>
#define DHTPIN 4
#define DHTTYPE DHT22
#define LED_OK 25
#define LED_ALERTA 27

const char* ssid = "Wokwi-GUEST";
const char* pass = "";
const char* mqtt_host =
"tu_cluster.s1.eu.hivemq.cloud";
const int mqtt_port = 8883;
const char* mqtt_user = "TU_USUARIO";
const char* mqtt_pass = "TU_PASSWORD";
const char* topic =
"salud/hospital/paciente001";
const char* topic_alerta =
"salud/hospital/alertas";
const char* paciente_id = "PAC-CARD-001";

WiFiClientSecure espClient;
PubSubClient mqtt(espClient);
DHT dht(DHTPIN, DHTTYPE);
int seq = 0;
int alertas_consecutivas = 0;

void setup() {
  Serial.begin(115200); dht.begin();
  pinMode(LED_OK, OUTPUT);
  pinMode(LED_ALERTA, OUTPUT);
  WiFi.begin(ssid, pass);
  while(WiFi.status() != WL_CONNECTED)
    delay(500);
  Serial.println("WiFi OK");
  espClient.setInsecure(); // En Wokwi (sin
  CA propia)

  mqtt.setServer(mqtt_host, mqtt_port);
}

void reconectar() {
  while(!mqtt.connected()) {
    if(mqtt.connect(paciente_id, mqtt_user,
mqtt_pass)) {
      Serial.println("MQTT TLS OK");
    } else {
      Serial.printf("Error: %d\n",
mqtt.state());
      delay(3000);
    }
  }
}

void loop() {
  if(!mqtt.connected()) reconectar();
  mqtt.loop();
  float t = dht.readTemperature();
  if(isnan(t)) { delay(2000); return; }

  // Simular signos vitales
  int bpm = 60 + random(0, 50); // 60-110
  int spo2 = 92 + random(0, 8); // 92-100
  seq++;

  // Lógica de alerta médica
  bool fuera_rango = (bpm > 100 || spo2 <
95);
  if(fuera_rango) {
    alertas_consecutivas++;
    digitalWrite(LED_OK, LOW);
    digitalWrite(LED_ALERTA, HIGH);
  } else {
    alertas_consecutivas = 0;
    digitalWrite(LED_OK, HIGH);
    digitalWrite(LED_ALERTA, LOW);
  }

  char json[300];
  snprintf(json, 300,
    "{\"paciente\":\"%s\", \"bpm\":%d, \"spo2\":%d, \"temp\":%.1f, \"alertas\":%d, \"seq\":%d}",
    paciente_id, bpm, spo2, t,
    alertas_consecutivas, seq);
  mqtt.publish(topic, json);
  Serial.println(json);

  // Si 3+ lecturas consecutivas fuera de
rango → ALERTA
  if(alertas_consecutivas >= 3) {
    char alert[200];
    snprintf(alert, 200,
    "{\"paciente\":\"%s\", \"tipo\":\"CRITICO\", \"bpm\":%d, \"spo2\":%d}",
    paciente_id, bpm, spo2);
    mqtt.publish(topic_alerta, alert);
    Serial.println("*** ALERTA MEDICA ***");
  }
  delay(5000);
}
```

Notas Clave

setInsecure()

En Wokwi para desarrollo (sin CA). En producción real usar setCACert() con el certificado raíz.

Puerto 8883 (TLS)

Obligatorio en datos médicos. Cumple normativa HIPAA y LGPD.

Lógica clínica:

bpm > 100 = taquicardia

spo2 < 95% = hipoxemia

3 lecturas seguidas para filtrar falsas alarmas.

Dos topics MQTT:

paciente: telemetría continua

alertas: solo eventos críticos

alertas_consecutivas

Filtro anti-ruido: solo dispara después de 3 lecturas anormales seguidas (15 seg).

LAB PBL • Código MicroPython – Wearable Médico



MicroPython – Equivalente con umqtt.simple + ssl

```
import network, dht, machine, time, json,
ssl, random
from umqtt.simple import MQTTClient

# Configuración
WIFI_SSID = 'Wokwi-GUEST'
MQTT_HOST = 'TU_CLUSTER.s1.eu.hivemq.cloud'
MQTT_PORT = 8883
MQTT_USER = 'TU_USUARIO'
MQTT_PASS = 'TU_PASSWORD'
TOPIC = 'salud/hospital/paciente001'
TOPIC_ALERT = 'salud/hospital/alertas'
PACIENTE_ID = 'PAC-CARD-001'

# Hardware
sensor = dht.DHT22(machine.Pin(4))
led_ok = machine.Pin(25, machine.Pin.OUT)
led_alert = machine.Pin(27, machine.Pin.OUT)

# WiFi
sta = network.WLAN(network.STA_IF)
sta.active(True); sta.connect(WIFI_SSID, '')
while not sta.isconnected(): time.sleep(0.5)
print('WiFi OK')

# MQTT con SSL/TLS
client = MQTTClient(
    client_id=PACIENTE_ID,
    server=MQTT_HOST,
    port=MQTT_PORT,
    user=MQTT_USER,
    password=MQTT_PASS,
    ssl=True,
    ssl_params={} # En producción incluir
    certificado
)
client.connect()
print('MQTT TLS OK')
```

```
# Variables
seq = 0
alertas_consecutivas = 0

while True:
    sensor.measure()
    t = sensor.temperature()
    seq += 1

    # Simular signos vitales
    bpm = 60 + random.randint(0, 50)
    spo2 = 92 + random.randint(0, 8)

    # Lógica de alerta
    fuera_rango = (bpm > 100 or spo2 < 95)
    if fuera_rango:
        alertas_consecutivas += 1
        led_ok.off(); led_alert.on()
    else:
        alertas_consecutivas = 0
        led_ok.on(); led_alert.off()

    # Telemetría
    payload = json.dumps({
        'paciente': PACIENTE_ID,
        'bpm': bpm, 'spo2': spo2,
        'temp': round(t, 1),
        'alertas': alertas_consecutivas,
        'seq': seq
    })
    client.publish(TOPIC, payload)
    print(payload)

    # Alerta crítica
    if alertas_consecutivas >= 3:
        alert = json.dumps({
            'paciente': PACIENTE_ID,
```

```
'tipo': 'CRITICO',
'bpm': bpm, 'spo2': spo2
})
client.publish(TOPIC_ALERT, alert)
print('** ALERTA MEDICA **')

time.sleep(5)
```

Equivalencias Arduino

Recordatorio:

MicroPython en Wokwi NO conecta a MQTT externo. Para Wokwi usar Arduino. Este código sirve para ESP32 físico.

Equivalencias clave:

WiFiClientSecure → ssl=True

Activa TLS en el constructor MQTTClient.

random(0,50) → random.randint(0,50)

snprintf() → json.dumps()

Comandos MicroPython clave:

MQTTClient(ssl=True): habilita TLS

client.publish(): envía mensaje

random.randint(a,b): número aleatorio

json.dumps(dict): serializa a JSON

LAB PBL · Código Python — Análisis y Alertas en Colab



Código Python — Análisis de signos vitales

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import paho.mqtt.client as mqtt
import ssl, json

# OPCIÓN A: Conectar al broker HiveMQ y suscribirse
# OPCIÓN B: Cargar histórico desde CSV exportado de Datacake

# 1. Cargar datos exportados de Datacake
df = pd.read_csv('paciente_historico.csv')
df['timestamp'] = pd.to_datetime(df['timestamp'])
print(f"Lecturas totales: {len(df)}")
print(f"Pacientes: {df['paciente'].unique()}")

# 2. Detectar anomalías por paciente
def detectar_alertas(df):
    df['taquicardia'] = df['bpm'] > 100
    df['bradicardia'] = df['bpm'] < 60
    df['hipoxemia'] = df['spo2'] < 95
    df['fiebre'] = df['temp'] > 37.5

    df['alerta'] = (df['taquicardia'] | df['bradicardia']
                  | df['hipoxemia'] | df['fiebre'])

    return df

df = detectar_alertas(df)
n_alertas = df['alerta'].sum()
print(f"Eventos de alerta: {n_alertas}")

# 3. Identificar episodios consecutivos (>3 lecturas)
df['grupo'] = (df['alerta'] != df['alerta'].shift()).cumsum()
episodios = df[df['alerta']].groupby('grupo').agg(
    inicio=('timestamp', 'min'),
    fin=('timestamp', 'max'),
    duracion_min=('timestamp',
        lambda x: (x.max() - x.min()).total_seconds() / 60),
    bpm_max=('bpm', 'max'),
    spo2_min=('spo2', 'min'),
    paciente=('paciente', 'first')
).reset_index(drop=True)
criticos = episodios[episodios['duracion_min'] >= 1]
print(f"Episodios CRITICOS (>1 min): {len(criticos)}")

# 4. Gráfico de signos vitales con alertas
fig, axes = plt.subplots(3, 1, figsize=(14, 8))
axes[0].plot(df['timestamp'], df['bpm'],
             color='#C62828', linewidth=0.8)
axes[0].axhline(y=100, linestyle='--',
               color='red', label='Taquicardia')
axes[0].axhline(y=60, linestyle='--',
               color='blue', label='Bradicardia')
axes[0].set_ylabel('BPM'); axes[0].legend()

axes[1].plot(df['timestamp'], df['spo2'],
             color='#1976D2', linewidth=0.8)
axes[1].axhline(y=95, linestyle='--', color='red')
axes[1].set_ylabel('SpO2 (%)')

axes[2].plot(df['timestamp'], df['temp'],
             color='#E8912D', linewidth=0.8)
axes[2].axhline(y=37.5, linestyle='--', color='red')
axes[2].set_ylabel('Temp (°C)')

plt.suptitle(f'Monitoreo Cardiológico | '
             f'{len(criticos)} episodios críticos')
plt.tight_layout(); plt.show()
```

Explicación Clave

Umbral médico:

bpm > 100: taquicardia

bpm < 60: bradicardia

spo2 < 95: hipoxemia

temp > 37.5: fiebre

Detección de episodios:

`groupby('grupo')`

Agrupar lecturas consecutivas anormales en "episodios". Cada episodio = 1 alerta.

Criterio crítico:

Episodios con duración > 1 minuto activan notificación al equipo médico.

Gráfico múltiple:

subplots(3,1) genera 3 gráficos en columna: BPM, SpO2 y temperatura, con sus umbrales clínicos marcados.

En el LAB se exporta el CSV histórico desde Datacake (botón Export).

7. RESUMEN, BIBLIOGRAFÍA Y CIERRE

Resumen – Ideas Clave de la Semana 7

4 IDEAS CLAVE QUE DEBES LLEVAR DE HOY

1

La salud conectada está transformando la medicina

Wearables, RPM, telemedicina y hospital inteligente generan ahorros, mejoran adherencia y salvan vidas. Apple Watch detecta fibrilación auricular con 98% de sensibilidad. La pandemia aceleró su adopción 10-15 años.

2

IoT tiene vulnerabilidades estructurales que deben gestionarse

Recursos limitados, credenciales por defecto, firmware sin actualizar, comunicación sin cifrar. Mirai infectó 600K dispositivos en 2016. Implantes médicos hackeados (St. Jude) cambiaron la regulación FDA.

3

Las 7 medidas mínimas son obligatorias en cualquier despliegue

Cambiar credenciales, cifrar TLS, autenticar, segmentar red, OTA seguro, logs, mínimo privilegio. STRIDE como modelo sistemático de análisis. Zero Trust como arquitectura recomendada.

4

La ética no es opcional: privacidad, sesgos e impacto social

GDPR, LGPD y normativa local definen el marco. Minimización, transparencia, consentimiento y anonimización son principios. Considerar siempre vigilancia, uso secundario, sesgos algorítmicos e impacto ambiental.



PRÓXIMOS PASOS: Semana 8: Estrategia, Modelos de Negocio y Presentación del Proyecto Integrador

Referencias



Bibliografía — Semana 7

Textos Base del Curso

Sicari, S. et al. (2015). Security, privacy and trust in IoT: the road ahead. Computer Networks.

Weber, R.H. (2010). IoT – New security and privacy challenges. Computer Law & Security Review.

Greengard, S. (2015). The Internet of Things. MIT Press. Capítulos sobre salud y privacidad.

Bahga, A. & Madiseti, V. (2014). Internet of Things: A Hands-On Approach. VPT.

Estándares y Casos

OWASP IoT Top 10. owasp.org/www-project-internet-of-things

Microsoft STRIDE Model. learn.microsoft.com/security/stride

FDA Cybersecurity Guidance for Medical Devices.

Regulación GDPR. gdpr.eu | LGPD. lgpd.com.br

Apple Heart Study. Stanford University (2019).

HiveMQ Cloud Documentation. hivemq.com/docs



¡MUCHAS GRACIAS!